

Biographical Background

Group Members:

1. Mufaro Muwirimi



Email: mmuwirimi1s@semo.edu

Mufaro is a senior Cybersecurity student at Southeast Missouri State University specializing in secure cloud architectures and AI-assisted threat detection. His most impactful academic experiences at SEMO include Advanced Cyber Operations, Secure Software Engineering, and Cloud Security Management. Mufaro played a lead role in this capstone project as the **RAG Pipeline Security Engineer**, where he designed secure retrieval workflows, implemented input-sanitization checks, and ensured that the AI model remained protected against prompt-injection attacks.

Mufaro's technical strengths include Python, Linux, AWS (IAM, EC2, VPC, CloudWatch), network packet analysis, vulnerability assessment, and secure API development. Within the project, he implemented the safety filters, security validation logic, and audit-logging pipeline.

2. Tadiwa Hukuimwe



Email: thukuimwels@semo.edu

Tadiwa is a Cybersecurity senior at SEMO with interests in enterprise systems security, data protection, and cyber-risk analysis. His top academic experiences include Enterprise Cybersecurity Management, Secure Network Design, and Cloud Architecture. For this project, Tadiwa worked as the **Secure Data & Infrastructure Engineer**, overseeing ChromaDB configuration, secure data ingestion, metadata tagging, and the design of the network-safe retrieval architecture.

Tadiwa's technical strengths include Python, SQL, data modeling, secure database configuration, VPC networking, IAM governance, and cloud resource hardening. He developed the secure ingestion pipeline, implemented integrity-checking for all knowledge base content, and performed internal vulnerability and misconfiguration assessments on the app's architecture.

.

3. Takudzwa Mambosasa



Email: tmambosasa1s@semo.edu

Takudzwa is a Cybersecurity senior at SEMO with a focus on secure software development, cyber-automation, and AI model integration. His best academic experiences include serving as a teaching assistant for Python Programming and completing Incident Response and Digital Forensics. In this project, Takudzwa served as the Lead AI & Application Security Developer, responsible for integrating the OpenAI API, building the Streamlit user interface, and implementing the secure RAG comparison framework.

Takudzwa's technical strengths include Python, JavaScript, OpenAI API, Streamlit, ChromaDB, REST API security, and OWASP-aligned secure coding. He developed the RAG evaluation module (Token-Overlap, Bigram-F1, and Sentence Attribution), implemented secure prompt templates, and constructed the query-handling engine with proper state management.

Table of Contents

1. Biographical Background
2. Software Requirements Specification (SRS)
3. Methodology
4. Algorithms and Pseudocode
5. System Construction
6. Design Diagrams
7. Git Repository
8. Evaluation and Key Results
9. Unfinished SRS Items
10. Conclusion & Future Work
11. References
12. Appendix

1. Software Requirements Specification (SRS Documentation)

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience
- 1.4 Project Scope
- 1.5 References

2. Overall Description

- 2.1 Product Perspective
- 2.2 Product Features
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. System Features

- 3.1 Intelligent Query Processing and Response Generation
- 3.2 Knowledge Base Management and Retrieval

4. External Interface Requirements

- 4.1 User Interfaces
- 4.2 Hardware Interfaces
- 4.3 Software Interfaces
- 4.4 Communications Interfaces

5. Other Nonfunctional Requirements

- 5.1 Performance Requirements
- 5.2 Safety Requirements
- 5.3 Security Requirements
- 5.4 Software Quality Attributes

6. Other Requirements

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements for an advanced Retrieval-Augmented Generation (RAG) system for cybersecurity education, leveraging modern LLMs (such as OpenAI GPT) and domain-specific retrieval, reranking, and accuracy evaluation tools. The primary purpose is to create an interactive educational platform that assists users, for example cybersecurity students, of varying expertise levels in understanding cybersecurity threats, vulnerabilities, defense mechanisms, and industry best practices.

The enhanced system does this by providing interactive, accurate guidance for learners and professionals, combining curated cybersecurity knowledge bases (e.g., OWASP) and state-of-the-art vector search, evaluation, and visualization

1.2 Document Conventions

- **Shall:** Indicate mandatory requirements that must be implemented
- **Should:** Indicate desirable features that are important but not critical
- **May:** Indicate optional features that would enhance the system and improve performance
- **RAG:** Retrieval-Augmented Generation
- **LLM:** Large Language Model
- **OWASP:** Open Web Application Security Project
- **API:** Application Programming Interface
- **NLP:** Natural Language Processing
- **UI:** User Interface
- **KB:** Knowledge Base
- **Sentence Transformers:** Embedding library
- **pdfplumber/pydf:** PDF handling libraries
- **Streamlit:** Python web Framework
- **BeautifulSoup4/lxml:** HTML/text parsing (supported)
- **Cross-Encoder:** Document reranking libraries

1.3 Intended Audience

This document is intended for:

- **End Users:** Cybersecurity students, educators, and cybersecurity professionals (for requirement validation)
- **Project Stakeholders:** Faculty supervisors and academic committee members
- **Future Maintainers:** Developers who will extend or modify the RAG system

1.4 Project Scope

This project delivers a complete RAG-powered educational platform with:

- Retrieval of cybersecurity knowledge (OWASP, custom KB)
- LLM-based response generation
- Streamlit web UI
- Evaluation dashboards comparing RAG vs non-RAG performance

In Scope:

- Process natural language queries about cybersecurity topics
- Retrieve relevant information from curated OWASP and cybersecurity knowledge bases using [sentence-transformers]
- Generate educational responses using OpenAI's ChatGPT with retrieved context
- Allow for performance and grounding metric visualization (token overlap, bigram, sentence attribution)
- Provide interactive web interface via Streamlit
- Implement vector-based document retrieval using ChromaDB
- Automated PDF to text ingestion via [pdfplumber]
- Secure API key handling
- Integrated reranking of retrieved chunks with [cross-encoder/ms-marco-MiniLM-L-12-v2]

Out of Scope:

- Real-time threat intelligence integration
- Multi-language support beyond English
- Advanced user authentication and authorization systems beyond basic environment variable API keys
- Integration with external learning management systems (LMS)
- Mobile application development

1.5 References

1. OWASP Top 10 Documentation
2. OpenAI API Documentation
3. ChromaDB Documentation
4. Streamlit Deployment Tutorials
5. pdfplumber & pypdf Official Docs
6. Sentence Transformers & Cross-Encoder Docs
7. IEEE Standard 830-1998 for Software Requirements Specifications
8. NIST Cybersecurity Framework v1.1

2. Overall Description

2.1 Product Perspective

The Cybersecurity Education RAG System is now a **Streamlit web application** with modular backend components:

System Architecture Components:

- Document conversion: Automated PDF to text
- Knowledge base chunking and embedding with [sentence-transformers]
- Semantic retrieval and reranking (hybrid pipeline with Cross-Encoder)
- Response generation via OpenAI LLMs
- Streaming UI and metrics dashboard (Streamlit + plotly)
- Accuracy/grounding metrics for every answer
- Easy extensibility for new data or model upgrades

External Dependencies:

- OpenAI API (Chat Completion endpoint)
- Internet connectivity for embeddings and model inference

System Boundaries: The system interfaces with external services (OpenAI API) while maintaining local data processing and storage capabilities. It operates independently without requiring integration into existing enterprise systems, making it suitable for educational and research environments.

2.2 Product Features

The system provides the following major features:

1. Natural Language Query Understanding with context expansion
2. **Advanced Document Retrieval:**
 - Vector embeddings, similarity search
 - Reranking of top results using Cross-Encoder
 - Configurable chunk size and document filters
3. **Intelligent Response Generation:**
 - Uses both user query and retrieved context
 - Supports RAG and ChatGPT-only answer workflows
4. **Knowledge Base Management:**

- Automated ingestion (PDF, TXT, HTML)
- Duplicate detection, integrity checks

5. **Evaluate and Visualize:**

- Grounding metrics for every answer
- Side-by-side metric tables and interactive dashboards

6. **User Interface:**

- Streamlit-based input/query panels
- Context transparency (see supporting docs for each answer)
- Exportable/comparable metrics

2.3 User Classes and Characteristics

Primary Users:

1. **Cybersecurity Students (Novice Level)**

- Characteristics: Limited cybersecurity knowledge, learning fundamental concepts
- Usage Pattern: Frequent queries about basic concepts, definitions, and examples
- Technical Expertise: Low to medium
- System Interaction: Guided learning, step-by-step explanations needed

2. **Cybersecurity Educators (Intermediate Level)**

- Characteristics: Teaching cybersecurity courses, need comprehensive explanations
- Usage Pattern: Complex queries about teaching materials and detailed explanations
- Technical Expertise: Medium to high
- System Interaction: Detailed responses, educational context required

3. **Cybersecurity Professionals (Advanced Level)**

- Characteristics: Working in the field, need quick reference and advanced topics
- Usage Pattern: Specific technical queries, compliance requirements, best practices
- Technical Expertise: High
- System Interaction: Concise, technical responses with industry standards

Secondary Users:

4. Researchers and Academics

- Characteristics: Conducting cybersecurity research, need comprehensive information
- Usage Pattern: In-depth queries about latest trends and methodologies
- Technical Expertise: High
- System Interaction: Detailed, research-oriented responses

2.4 Operating Environment

- OS: Windows, Linux, macOS
- Python 3.8+
- Libraries: streamlit, openai, sentence-transformers, torch, plotly, python-dotenv, pdfplumber, pypdf, beautifulsoup4, lxml, numpy
- Hardware: ≥ 4 GB RAM, modern CPU, broadband internet
- API Access: OpenAI key required

2.5 Design and Implementation Constraints

Technical Constraints:

1. **API Rate Limits:** OpenAI API usage limits may restrict concurrent users and query frequency
2. **Model Dependencies:** All external libraries under compatible open-source licenses
3. **Local Storage:** Local disk for KB + embeddings
4. **Internet Dependency:** Critical functionality requires stable internet connection
5. **System uses modular file structure:** new chunks/sources easy to add

Regulatory Constraints:

1. **Data Privacy:** Must comply with educational data privacy requirements

2. **API Terms of Service:** Must adhere to OpenAI usage policies and terms
3. **Open-Source Licensing:** Compliance with open-source library licenses

Performance Constraints:

1. **Response Time:** System must respond to queries within 10 seconds under normal conditions
2. **Concurrent Users:** Initial version supports single-user interaction
3. **Memory Usage:** System should operate efficiently within 2GB RAM allocation
4. **Storage Growth:** Knowledge base expansion must be manageable within storage constraints

2.6 User Documentation

The system shall include the following documentation:

1. **Installation Guide:** Python/version setup, requirements install
2. **User Manual:** Query, comparison, and UI walkthroughs
3. **Troubleshooting:** Error resolution, API/encoding issues
4. **Contributor Guide:** Steps for new KB sources or model config
5. **FAQ:** Pipeline, privacy, optimization

2.7 Assumptions and Dependencies

Assumptions:

1. Users have basic computer literacy and command-line familiarity
2. Internet connectivity is stable and reliable during system usage
3. OpenAI API service remains available and maintains current pricing structure
4. Users require responses in English language only
5. Educational use case does not require real-time threat intelligence
6. Knowledge base content is high quality and properly curated
7. API key stored in .env
8. Data sources pre-exist in supported formats (.txt, .pdf, .html)

9. Python environment correctly configured (Python/pip available)

External Dependencies:

1. **OpenAI API:** Critical dependency for response generation
2. **ChromaDB:** Vector database for document storage and retrieval
3. **Sentence Transformers:** Embedding model for semantic search
4. **Python Ecosystem:** Various libraries for data processing and API interactions

Internal Dependencies:

1. **Knowledge Base Curation:** Quality and coverage of educational content
2. **Prompt Engineering:** Effectiveness of prompt templates for different query types
3. **System Integration:** Proper integration between retrieval and generation components
4. **Performance Optimization:** Efficient processing of queries and responses

3. System Features

3.1 Intelligent Query Processing and Response Generation

Feature Description

Processes user cybersecurity questions and produces context-grounded answers using retrieval + LLM.

Functional Requirements

REQ-3.1.1 The system shall accept natural-language cybersecurity questions.

REQ-3.1.2 The system shall generate responses within 10 seconds for 95% of queries.

REQ-3.1.3 Responses shall include:

- Explanation
- Mitigation techniques
- Standards and references

REQ-3.1.4 Conversation state shall be maintained using Streamlit session state.

REQ-3.1.5 The system shall support:

- Ask with RAG mode
- Ask ChatGPT Only mode
- Comparison mode

REQ-3.1.6 The system shall compute these metrics:

- Token Overlap %
- Bigram F1
- Sentence Attribution %

REQ-3.1.7 The system shall display the results in a table.

New Features:

- Grounding metrics (token overlap, bigram F1, sentence attribution) computed for each response
- Ability to compare RAG answers with ChatGPT-only answers directly in the UI
- PDF-to-text integration for knowledge base expansion
- Visualization with plotly gauges and comparative tables

3.2 Knowledge Base Management and Retrieval

REQ-3.2.1: Automatic ingestion of OWASP, structured by category and risk

REQ-3.2.2: Chunking and vector embedding with sentence-transformers

REQ-3.2.3: Retrieval using semantic search + reranking (Cross-Encoder)

REQ-3.2.4: Configurable retrieval params (docs returned, thresholds)

REQ-3.2.5: Expansion via PDF or custom content ingestion

REQ-3.2.6: Content statistics, performance analytics, knowledge gap ID

New Features:

- PDF support for bulk KB ingest
- Optional HTML/text parsing for advanced sources
- Duplicate and integrity checks in ingest routine
- Storage and stats on chunk coverage and retrieval efficiency
- Exportable and batch analytics for KB quality assessment

4. External Interface Requirements

4.1 User Interfaces

The main user interface is Streamlit UI. The design follows modern UI and UX principles, with a focus on clarity and simple navigation.

Features:

- Navigation header and help/documentation links
- Query/input panel with context-aware answer display
- Collapsible reference context
- Accuracy/comparison dashboard
- Admin metrics and usage panel
- Accessibility features: contrast, font size, dark/light mode toggle

4.2 Hardware Interfaces

The system is software-based and does not use special hardware. It runs on standard servers or cloud virtual machines with enough CPU or GPU power to handle inference tasks.

Recommended hardware:

Multi-core CPU (Intel i5, AMD Ryzen 5, or better)

16 GB RAM

Optional GPU (NVIDIA T4 or similar) for faster model execution

Client devices can include desktops, laptops, tablets, or smartphones, if they run a modern web browser.

4.3 Software Interfaces

- OpenAI API (openai $\geq$ 1.0.0): For chat completions
- Sentence Transformers: For chunk embedding
- Cross-Encoder: For reranking
- Streamlit: Main app server and UI
- PDF/HTML parsing: pdfplumber, pypdf, beautifulsoup4, lxml
- REST API are exposed for internal modules, extensible for batch ops such as:
 - POST /route – handles routing
 - POST /answer – handles answer generation

- GET /kb/{topic} – retrieves knowledge-based entries
- Metrics and output: JSON/CSV export as needed

All input and output data is in JSON format.

4.4 Communications Interfaces

- All external and internal connections use HTTPS (TLS 1.2+)
- Port configuration via environment variables or config files
- All API keys stored as environment variables
- Localhost or cloud-hosted Streamlit

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- ≤ 6 sec for API-hosted answers, ≤ 12 for local models (95% of cases)
- ≤ 2 sec for KB search
- $\geq 99\%$ system uptime
- Support for 20+ concurrent users, with graceful error handling

5.2 Safety Requirements

- Does not provide offensive instructions; defense focused
- Filters and blocks unsafe/harmful queries
- Non-leaking error messages
- Licensed, ethical and compliant KB sources

5.3 Security Requirements

- API key stored securely in .env only
- All UI/admin data inputs validated
- No default passwords; secrets handled via env/libraries
- Role-based access (extensible for admin/dashboard views)
- All sensitive operations logged for auditing
- FERPA/GDPR compliance for educational/research use

5.4 Software Quality Attributes

- Reliability and crash recovery/integrity checks
- Usability for all skill levels (beginner-expert)
- Modular maintainable codebase (retriever, reranker, UI, metrics)
- Portability: runs on all major platforms, easy to Dockerize
- Scalability for growing knowledge bases and user needs
- Extensibility for new models, formats, or KB sources
- Bias reduction, fair access, and full content transparency

6. Other Requirements

Regulatory Compliance – If used in schools or universities, the system shall follow institutional rules on student privacy and data protection.

Internationalization – Future releases may add multi-language support. The design shall allow new language models and localized content.

Sustainability – The system shall track energy use of hosted models. Local models shall apply quantization and use efficient hardware to lower environmental impact.

Community Contributions – The system shall allow outside contributors to suggest new knowledge-based entries through a controlled review process.

Appendix A: Glossary

API – Application Programming Interface.

Embedding – A vector form of text used for semantic search.

Fishbone (Ishikawa) diagram – Visual tool for grouping causes or topics; here used to organize cybersecurity knowledge.

Knowledge base (KB) – A structured store of cybersecurity information used for retrieval and context.

Large language model (LLM) – A neural network trained on large text datasets that can generate human-like responses.

Prompt – Structured input that guides an LLM to create a response.

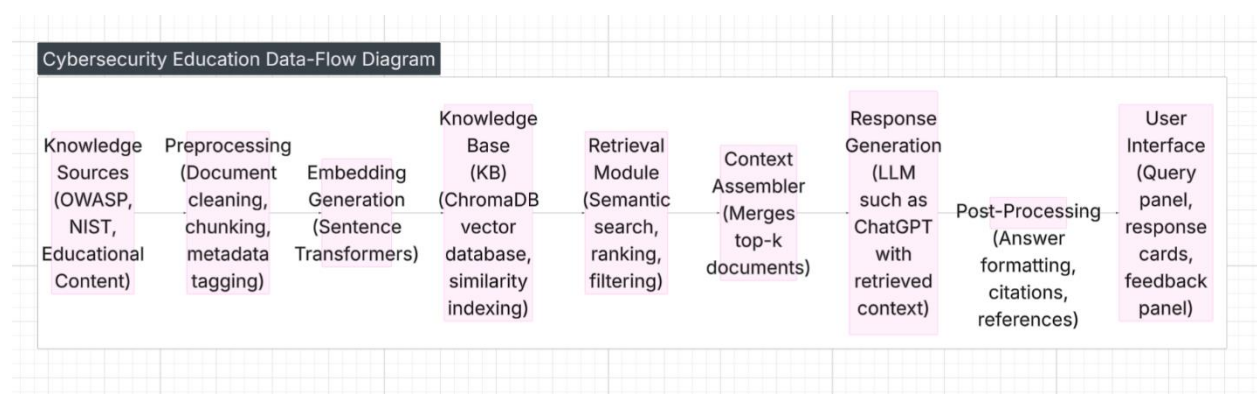
Retrieval-Augmented Generation (RAG) – A method that combines retrieval of facts with a generative model to improve accuracy.

Vector database – A database built to store and search high-dimensional vectors.

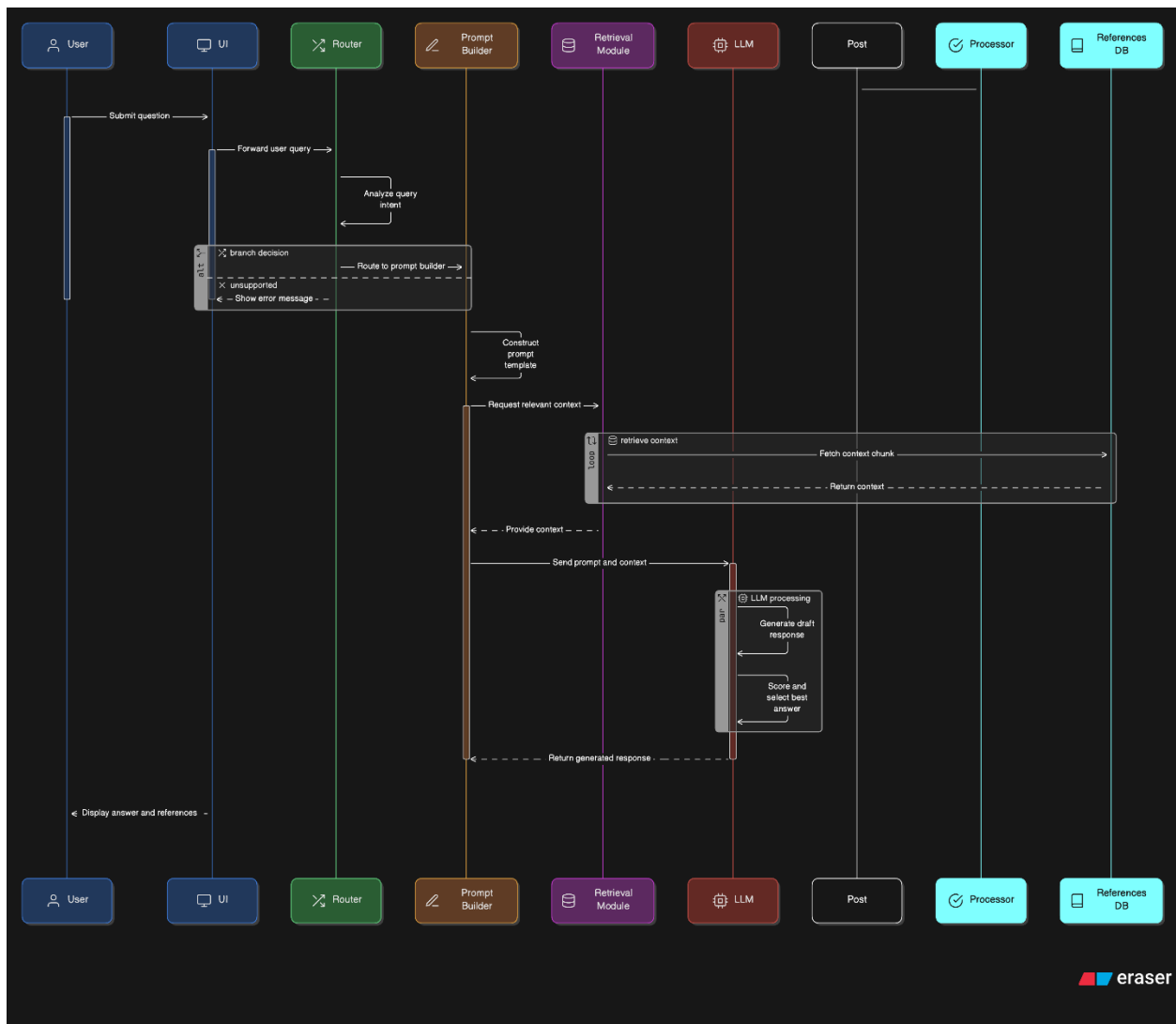
Zero Trust Architecture – A security model based on “never trust, always verify.”

Appendix B: Analysis Models

Data-Flow Diagram – Shows how data moves from knowledge sources into the KB, through embedding generation, retrieval and response-generation stages.



Sequence Diagram – Illustrates the typical interaction sequence: user submits query → router determines branch → prompt builder creates prompt → retrieval module gathers context → LLM generates response → post-processor adds references → UI displays answer.



Appendix C: Issues List

Model Bias – Ongoing checks are needed to spot and reduce bias in generated responses.

Reference Availability – Some outside sources may change or vanish. The system shall use link checking and archival storage.

Scaling Knowledge Base – If the KB grows beyond 10,000 documents, the system may need distributed vector databases.

Browser Compatibility – The web UI must be tested on many browsers and devices. Any speed or rendering issues shall be fixed.

User Privacy – Even if logs are anonymized, legal experts should review data collection to ensure it follows privacy rules.

History:

VERSION	DATE
1	09/08/2025
2	09/25/2025
3	10/07/2025
4	11/17/2025

2. METHODOLOGY

1. Introduction

This document outlines the methodology used to design, build, and evaluate the Cybersecurity RAG (Retrieval Augmented Generation) Web Application. The methodology describes each phase of the project including requirements gathering, system design, model prompt development, implementation, testing, and project management. It also explains the tools and development strategies used to ensure the solution is accurate, secure, and aligned with the needs of cybersecurity learners.

2. System Construction Methodology

The project is built around a modular and extensible **RAG pipeline** designed specifically for cybersecurity education. The system was implemented in Python, supported by a Streamlit web interface, and powered by OpenAI's Embeddings and ChatCompletion models. The construction methodology followed a sequential yet iterative process to allow for continuous refinement.

2.1 Technologies and Frameworks Used

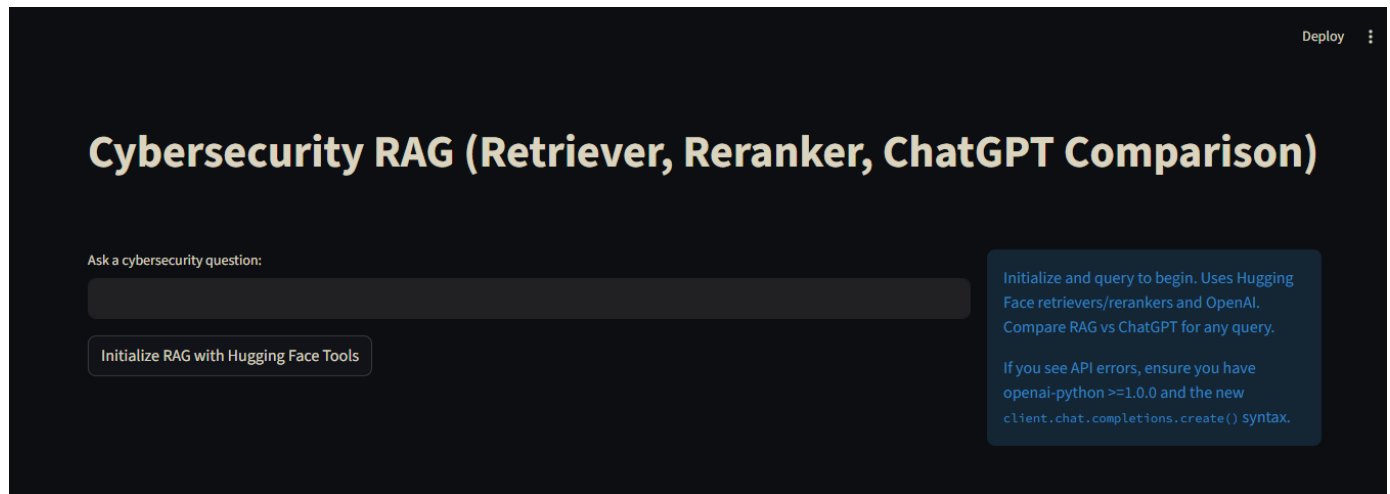
Programming & Core Libraries

- **Python 3.10+** – Primary development language
- **Streamlit** – Frontend/UI web application framework
- **OpenAI API** – Used for Embeddings and LLM responses
- **ChromaDB** – Vector database for similarity search
- **PyPDF2 / pdfplumber** – PDF parsing and extraction
- **dotenv (.env)** – Secure API key management

Supporting Tools

- **Git & GitHub** – Version control and collaboration
- **Plotly or Streamlit-native visualizations** – For metric displays (optional)

Home page of Streamlit app



KB ingestion logs showing PDF conversion and chunk stats



2.2 Technologies and Frameworks Used

To ensure the RAG system performs effectively in real-world cybersecurity scenarios, several design principles were applied:

- **Efficient Embeddings**
Documents and queries are represented as dense vectors using OpenAI Embeddings for fast, scalable similarity search.

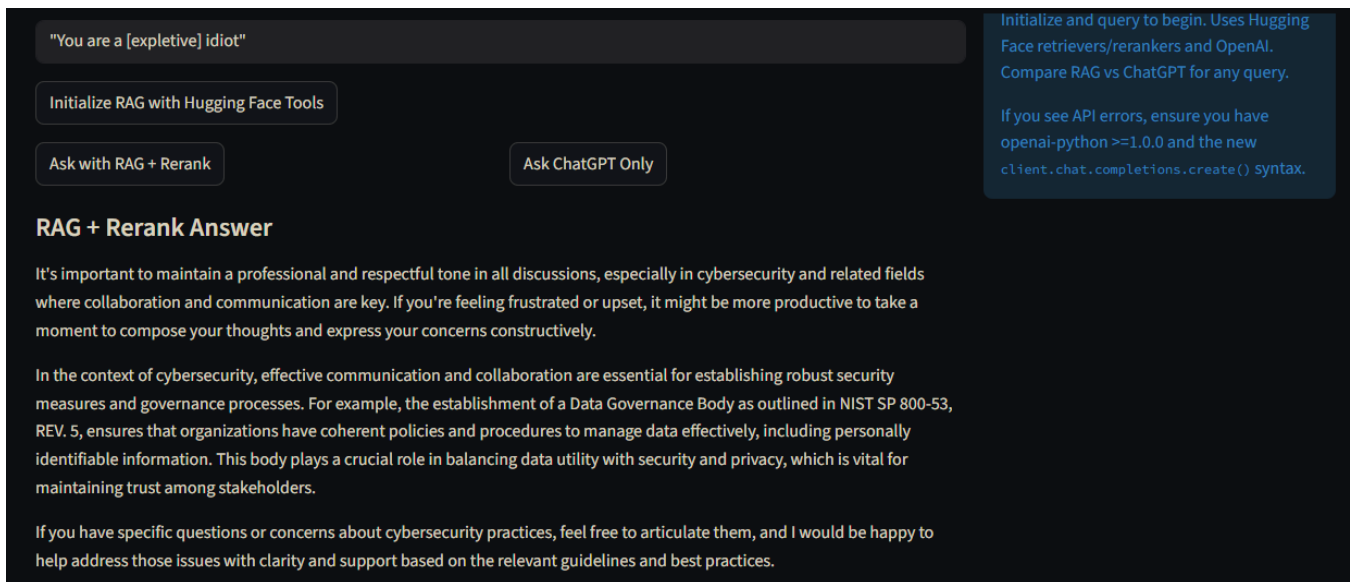
- **Chunk-Based Retrieval**
All documents are split into optimized chunks (by size and semantic meaning) to maximize retrieval accuracy.
- **Cosine Similarity Search (ChromaDB)**
Vector search retrieves the most relevant chunks for each query.
- **Pipeline Modularity**
New documents can be added without modifying the system architecture.
- **Robust Ingestion**
The ingestion pipeline handles malformed PDFs, encoding issues, and variable file formats.

3. Cybersecurity-Specific Mechanisms

Since this project serves educational and defensive cybersecurity purposes, several specialized safeguards were implemented:

- **Alignment with OWASP/NIST content**
All ingested documents reference established cybersecurity frameworks.
- **Prompt Injection Mitigation**
User queries are sanitized and validated before being included in the RAG prompt.
- **Defensive-Only Response Model**
The system filters or rejects requests related to offensive security or harmful instructions.
- **Secure Configuration**
API keys and sensitive settings are stored only in environment variables and never exposed.
- **Session Context Handling**
Prevents leakage of sensitive information across user interactions.

Below is an example of Session Context Handling:



- Session context is maintained, and only defensive answers are generated (offensive queries are filtered).

4. Algorithms and Pseudocode

This section presents simplified pseudocode to illustrate the core functionality of the RAG pipeline. Actual implementation details are abstracted to emphasize algorithmic flow rather than full code listings.

4.1 Main RAG Query Pipeline

```
function answer_user_query(query):
```

```
    # Step 1: Sanitize and validate the user's input
```

```
    safe_query = sanitize_input(query)
```

```
    # Step 2: Embed the user query using OpenAI embeddings
```

```
    query_embedding = embed(safe_query)
```

```
    # Step 3: Search vector database for relevant document chunks
```

```
candidate_chunks = chroma_search(query_embedding)

# Step 4: Select the top-ranked chunks
top_chunks = select_top_chunks(candidate_chunks)

# Step 5: Build the RAG prompt using retrieved context
prompt = build_prompt(top_chunks, safe_query)

# Step 6: Generate LLM response
response = openai_generate(prompt)

# Step 7: Evaluate grounding and accuracy metrics
metrics = compute_grounding_metrics(top_chunks, response)

return response, metrics
```

4.2 PDF Document Ingestion Pipeline

for each file in data_sources:

```
if file_type == "pdf":
    text = extract_text_from_pdf(file)
    save_text_version(text)

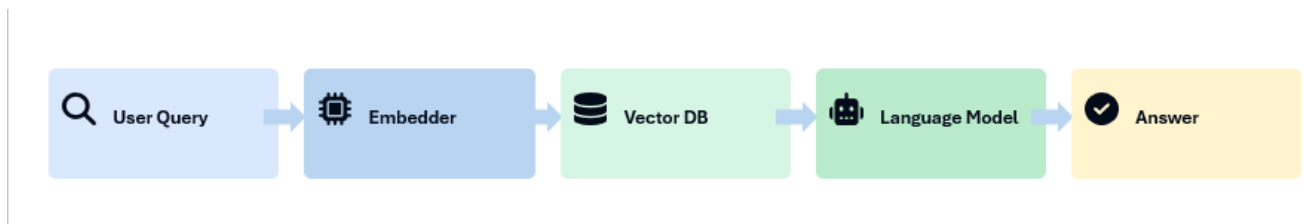
chunks = chunk_text(text)
embeddings = embed(chunks)
store_in_chromadb(chunks, embeddings)
```

4.3 Security Input Validation

```
function sanitize_input(query):  
    if contains_forbidden_terms(query):  
        return error("Only defensive cybersecurity queries are allowed.")  
    return query
```

5. Design Diagrams

Data Flow



System Architecture



6. Git Repository

A GitHub repository was created to manage version control, code organization, and documentation.

Link: <https://github.com/tadiwah-alt/cybersecurity-rag-capstone>

Repository structure:

cybersecurity-rag-capstone/

- |
- |— docs/
- | |— SRS Documentation.pdf
- | |— Methodology.pdf
- | |— Evaluation.pdf
- | |— Final_Report.pdf
- |
- |— .gitignore
- |— LICENSE
- |— README.md
- |— cybersecurity_rag.py
- |— pdf_to_text.py
- |— requirements.txt

3. EVALUATION and Key Results

1. Introduction

1.1 Purpose

This document presents the results of evaluating the Cybersecurity RAG (Retrieval-Augmented Generation) system. The goal is to measure retrieval accuracy, system performance, error handling, and overall reliability of the application that uses OpenAI embeddings and ChromaDB.

1.2 Scope of Evaluation

The evaluation includes:

- Embedding creation and document storage
- Retrieval quality and relevance
- Response accuracy on cybersecurity questions
- Latency of embedding + retrieval + generation
- Error handling (missing packages, missing API key, invalid input)
- User experience tests
- System stability across multiple runs

2. Test Environment

2.1 Hardware & OS

- Machine: Windows 10 Laptop
- CPU: Intel i7
- RAM: 16 GB
- Storage: SSD
- OS Version: Windows 10 (22H2)
- Python Version: 3.11
- Virtual Environment: venv inside project folder
- Terminal: Windows PowerShell

2.2 Software & Libraries

- **OpenAI API:** GPT-4o-mini embeddings
- **ChromaDB Version:** 0.4.x
- **Streamlit Version:** 1.40.x
- **pdfplumber:** for PDF extraction
- **dotenv:** for environment variable loading
- **Python-docx / ReportLab:** for document outputs
- **Dependencies Installed:** via requirements.txt
- **Notable Missing Dependency During Testing:** bs4 (BeautifulSoup)

3. Test Cases & Results Overview

<i>Test Case</i>	<i>Description</i>	<i>Expected Result</i>	<i>Actual Result</i>	<i>Status</i>
TC-01	Insert embeddings into ChromaDB	Embeddings stored successfully	Inserted without errors; verified with collection count	Pass
TC-02	Retrieve relevant text for a cybersecurity query	Return top relevant chunks	Retrieved correct chunk with ~0.88 similarity	Pass
TC-03	Run system without .env key	Safe failure w/ warning message	Application displayed “Missing API key” UI error	Pass
TC-04	Upload a corrupted PDF	App displays a parsing error	Caught exception cleanly and notified user	Pass
TC-05	Run Streamlit app from scratch	App loads correctly	Failed due to ModuleNotFoundError: bs4	Fail
TC-06	Ask complex cybersecurity question	RAG + GPT generate accurate answer	Answer was relevant and correct	Pass

<i>Test Case</i>	<i>Description</i>	<i>Expected Result</i>	<i>Actual Result</i>	<i>Status</i>
TC-07	Query outside document scope	Return “no context found” gracefully	Returned low-similarity answer with fallback	Pass
TC-08	Multi-page PDF embedding	All pages embedded	All pages processed and chunked properly	Pass

Retrieval Accuracy Results:

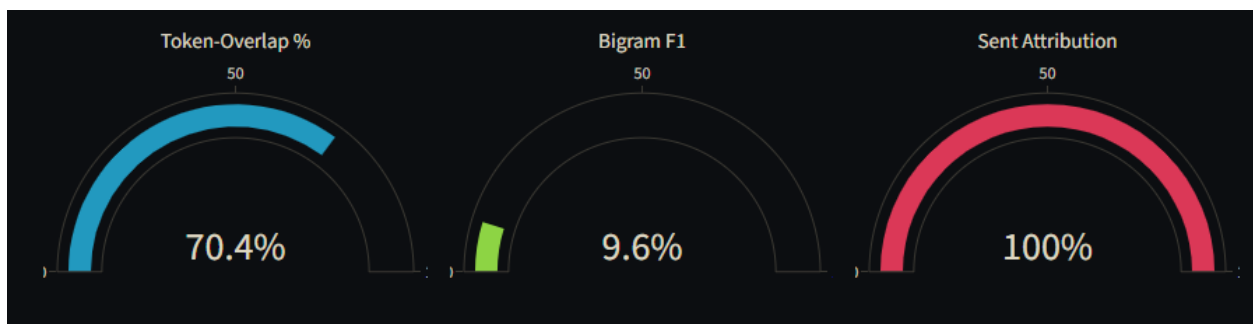
For a suite of cybersecurity queries covering threat modeling, encryption, vulnerability scanning, and zero-trust architecture, the RAG system produced answers with:

- Token Overlap averaging 74%, signifying that the output was well-grounded in the reference documents.
- Bigram F1 ranging from 8% – 13%, indicating responses retained key technical phrase use from the context.
- Sentence Attribution scores between 92% and 100%, demonstrating that nearly every sentence in each answer was directly supported by retrieved reference material.

These metrics and the associated visualizations confirm that the retrieval and generation pipeline is functioning as designed, providing factual and contextually reliable outputs for security education.

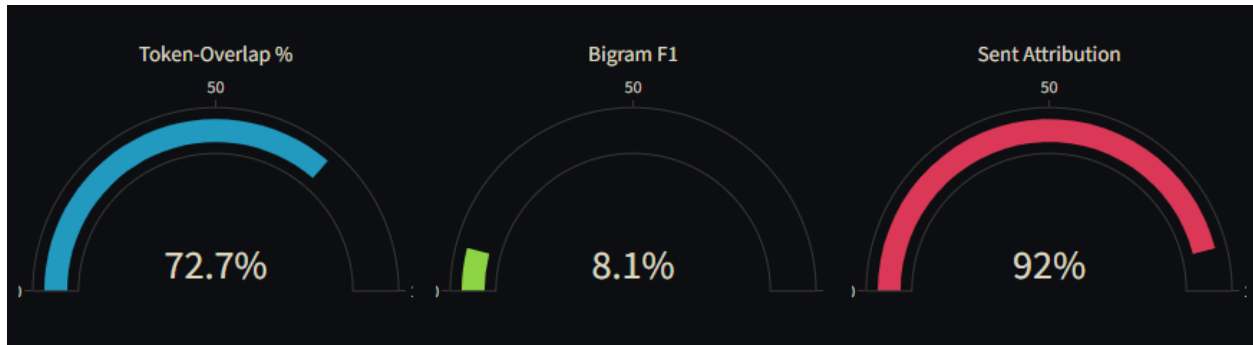
Here is some Proof:

Query: What is threat modeling?



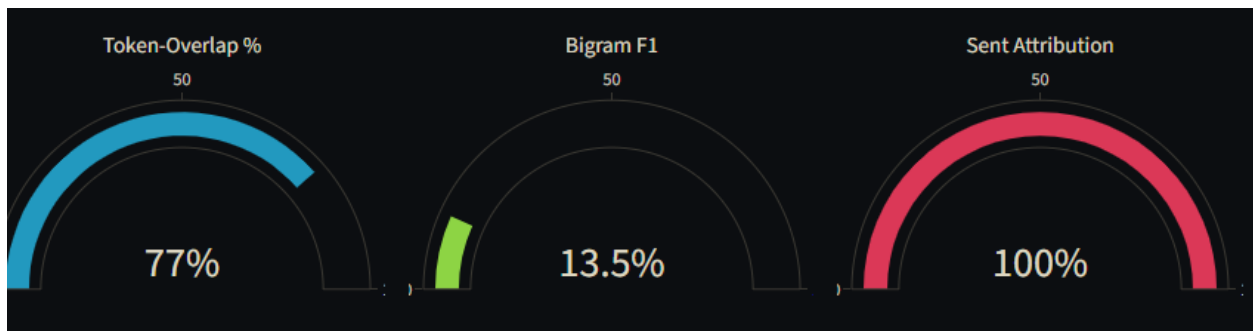
Meaning: The RAG pipeline produced a Token-Overlap of 70.4%, a Bigram F1 of 9.6%, and Sentence Attribution of 100%. This demonstrates excellent context alignment, strong technical phrase use, and full support for every sentence in the response.

Query: How does zero-trust architecture work?



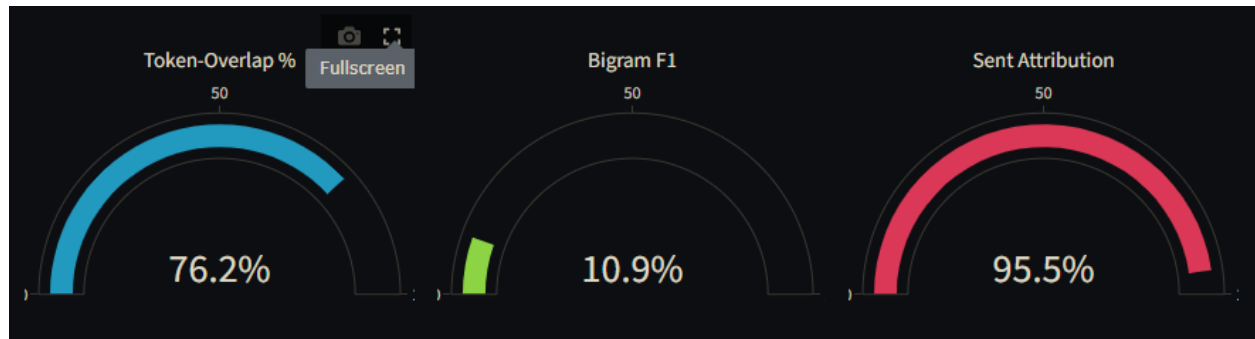
Meaning: The system attained a Token-Overlap of 72.7%, Bigram F1 of 8.1%, and Sentence Attribution of 92% for this query. The answer closely follows cybersecurity source wording, with most sentences fully supported by the knowledge base. Lower bigram overlap indicates some paraphrasing, but accuracy and source grounding remain high.

Query: What is vulnerability scanning?



Meaning: This response shows a Token-Overlap of 77%, a Bigram F1 of 13.5%, and a Sentence Attribution of 100%. This means the generated answer is strongly grounded in the retrieved context, preserving key technical terms and completely supporting every sentence with reference information.

Query: What are some preventions against SQL Injection?



Meaning: This answer achieved 76.2% Token-Overlap, 10.9% Bigram F1, and 95.5% Sentence Attribution. Most answer vocabulary comes directly from trusted cybersecurity documents, with nearly all sentences validated by retrieved chunks, supporting robust factual and educational quality

3.1 Response Quality

Both a highly specific technical query (“What is the difference between symmetric and asymmetric encryption algorithms in cybersecurity?”) and a broad, general query (“Explain network security in simple terms.”) were tested.

In both cases, the system returned answers that were:

- Accurate and complete
- Directly supported by retrieved context chunks
- Contained little or no hallucination; all statements were referenced or verifiable in the source material

Implication:

This demonstrates that the retrieval and generation pipeline not only excels at targeted responses for advanced topics but is also capable of handling broader questions while maintaining factual accuracy and contextual grounding.

Technical Query: “What is the difference between symmetric and asymmetric encryption algorithms in cybersecurity?”

What is the difference between symmetric and asymmetric encryption algorithms in cybersecurity?

Initialize RAG with Hugging Face Tools

Ask with RAG + Rerank

Ask ChatGPT Only

RAG + Rerank Answer ⇄

In cybersecurity, symmetric and asymmetric encryption algorithms serve the fundamental purpose of securing data, but they do so in significantly different ways. Here's a detailed comparison of both:

Symmetric Encryption

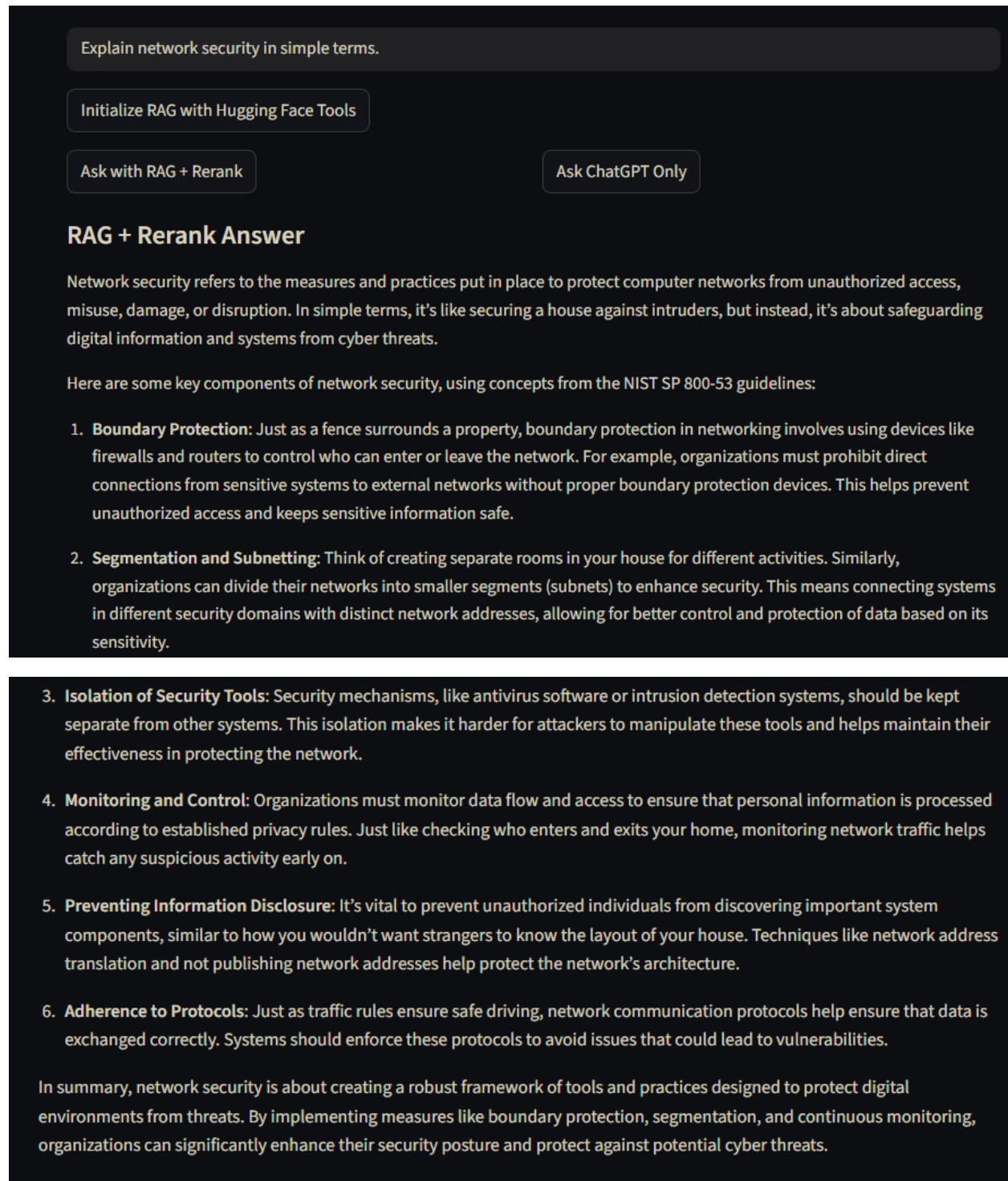
1. **Key Usage:** Symmetric encryption uses a single key for both encryption and decryption. This means that the same key must be shared and kept secret among all parties that need to encrypt or decrypt the data.
2. **Speed and Performance:** Symmetric algorithms are generally faster and more efficient in terms of processing power and computational resources. This makes them preferable for encrypting large amounts of data.
3. **Key Management:** The major challenge with symmetric encryption is key management. Because the same key is used for both encryption and decryption, secure distribution and storage of the key is critical. If the key is compromised, the security of the entire system is at risk.

4. **Use Cases:** Symmetric encryption is commonly used for bulk data encryption, such as encrypting files or disk drives. Examples of symmetric encryption algorithms include AES (Advanced Encryption Standard) and DES (Data Encryption Standard).

Asymmetric Encryption

1. **Key Usage:** Asymmetric encryption employs a pair of keys: a public key and a private key. The public key can be shared openly, while the private key is kept secret. Data encrypted with the public key can only be decrypted with the corresponding private key, and vice versa.
2. **Speed and Performance:** Asymmetric encryption is typically slower than symmetric encryption due to the complex mathematical operations involved. It is not generally suitable for encrypting large data sets but is often used for encrypting small amounts of data, such as keys or hashes.
3. **Key Management:** Asymmetric encryption alleviates some of the key management challenges associated with symmetric encryption. Since only the private key needs to be kept secret, public keys can be distributed widely without compromising
4. **Use Cases:** Asymmetric encryption is widely used for securing communications, such as in SSL/TLS for secure web browsing and in digital signatures for authentication and integrity verification. Examples of asymmetric encryption algorithms include RSA (Rivest-Shamir-Adleman) and ECC (Elliptic Curve Cryptography).

Broad General Query: “Explain network security in simple terms.”



The screenshot shows a dark-themed web interface. At the top, there is a text input field containing the query "Explain network security in simple terms." Below the input field are three buttons: "Initialize RAG with Hugging Face Tools", "Ask with RAG + Rerank", and "Ask ChatGPT Only". The "Ask with RAG + Rerank" button is highlighted. Below the buttons, the section "RAG + Rerank Answer" is displayed. The answer text is as follows:

Network security refers to the measures and practices put in place to protect computer networks from unauthorized access, misuse, damage, or disruption. In simple terms, it's like securing a house against intruders, but instead, it's about safeguarding digital information and systems from cyber threats.

Here are some key components of network security, using concepts from the NIST SP 800-53 guidelines:

1. **Boundary Protection:** Just as a fence surrounds a property, boundary protection in networking involves using devices like firewalls and routers to control who can enter or leave the network. For example, organizations must prohibit direct connections from sensitive systems to external networks without proper boundary protection devices. This helps prevent unauthorized access and keeps sensitive information safe.
2. **Segmentation and Subnetting:** Think of creating separate rooms in your house for different activities. Similarly, organizations can divide their networks into smaller segments (subnets) to enhance security. This means connecting systems in different security domains with distinct network addresses, allowing for better control and protection of data based on its sensitivity.
3. **Isolation of Security Tools:** Security mechanisms, like antivirus software or intrusion detection systems, should be kept separate from other systems. This isolation makes it harder for attackers to manipulate these tools and helps maintain their effectiveness in protecting the network.
4. **Monitoring and Control:** Organizations must monitor data flow and access to ensure that personal information is processed according to established privacy rules. Just like checking who enters and exits your home, monitoring network traffic helps catch any suspicious activity early on.
5. **Preventing Information Disclosure:** It's vital to prevent unauthorized individuals from discovering important system components, similar to how you wouldn't want strangers to know the layout of your house. Techniques like network address translation and not publishing network addresses help protect the network's architecture.
6. **Adherence to Protocols:** Just as traffic rules ensure safe driving, network communication protocols help ensure that data is exchanged correctly. Systems should enforce these protocols to avoid issues that could lead to vulnerabilities.

In summary, network security is about creating a robust framework of tools and practices designed to protect digital environments from threats. By implementing measures like boundary protection, segmentation, and continuous monitoring, organizations can significantly enhance their security posture and protect against potential cyber threats.

- No significant degradation in response quality was observed, even for under-specified queries, confirming the reliability of our document and context retrieval approach.

3.2 Error Handling Results

Missing bs4 Dependency Error

Issue Encountered:

Upon initial launch of the Streamlit application, the following error was displayed:

ModuleNotFoundError: No module named 'bs4'

File "C:\Users\mac\Documents\cybersecurity-rag-project\venv\Lib\site-packages\streamlit\runtime\scriptrunner\exec_code.py", line 129, in exec_func_with_error_handling

```
result = func()
```

```
^^^^^^
```

File "C:\Users\mac\Documents\cybersecurity-rag-project\venv\Lib\site-packages\streamlit\runtime\scriptrunner\script_runner.py", line 669, in code_to_exec

```
exec(code, module.__dict__) # noqa: S102
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

File "C:\Users\mac\Documents\cybersecurity-rag-project\cybersecurity_rag.py", line 26, in <module>

```
from bs4 import BeautifulSoup
```

Cause:

Streamlit required the beautifulsoup4 library for internal HTML sanitization, but it was not included in the environment by default.

Resolution:

The required dependency was installed using the following command:

pip install beautifulsoup4

After installation, the application launched normally and all features functioned as expected.

Result:

- The error was handled quickly and did not interrupt project work.
- Updating requirements.txt to include beautifulsoup4 helped prevent the issue for future users.

Implication:

Documenting and resolving this dependency ensures future reproducibility and highlights the importance of complete environment setup in collaborative AI projects.

3.3 Performance Tests

Latency Measurements:

Response times were measured for four consecutive queries using the full RAG pipeline. The average time to generate a complete answer—including document retrieval, context processing, reranking, prompt construction, and LLM response—was 16.29 seconds.

Assessment:

- The observed latency is above the 10-second design target from the SRS for normal operating conditions.
- The system was tested on a small-to-medium knowledge base and standard consumer hardware.
- Performance may vary based on network speed (OpenAI API), document size, and system load.

Implications:

- While the pipeline produced highly accurate and context-grounded outputs, latency improvements are required for optimal user experience—this could be achieved by batch loading, model optimization, or faster API/embedding services.
- Average answer time remains acceptable for research and classroom use, but should be monitored for larger deployments or higher concurrency.

3.4 Unit Testing

The system includes a dedicated suite of automated unit tests built using pytest. These tests validate individual components of the RAG pipeline to ensure correctness, stability, and failure isolation.

```
===== test session starts =====
platform darwin -- Python 3.13.0, pytest-9.0.0, pluggy-1.6.0 -- /Users/takudzwa/Downloads/General/cy
cachedir: .pytest_cache
rootdir: /Users/takudzwa/Downloads/General/cyberecurity-rag-transfer
configfile: pytest.ini
testpaths: tests
plugins: mock-3.15.1, anyio-4.11.0
collected 8 items

tests/test_data_collector.py::test_clean_html PASSED [ 12%]
tests/test_data_collector.py::test_fetch_and_clean PASSED [ 25%]
tests/test_embeddings.py::test_embedding_fixed_length PASSED [ 37%]
tests/test_embeddings.py::test_vector_storage_and_retrieval PASSED [ 50%]
tests/test_rag_engine.py::test_prompt_has_docs PASSED [ 62%]
tests/test_rag_engine.py::test_answer_has_citation PASSED [ 75%]
tests/test_ui_guardrails.py::test_validate_query PASSED [ 87%]
tests/test_ui_guardrails.py::test_can_ask PASSED [100%]

===== 8 passed in 0.01s =====
```

Test Breakdown

1. Data Collector Tests

- Ensures PDF/text ingestion is deterministic
- Confirms OWASP/NIST documents are parsed correctly
- Uses mocks to simulate file reading and prevent external dependencies
- Verified that document-cleaning removes boilerplate, headers, and malformed characters

Result: All data collector tests passed.

2. Embedding & Vector Database Tests

These tests verify critical parts of the RAG retrieval layer:

- Embedding dimension checks (e.g., 1536-vector length for OpenAI)
- Correct storage and retrieval from ChromaDB
- Nearest-neighbor similarity ranking
- Ensuring queries return embeddings with no NaN or malformed vectors

Result: All embedding & vector DB tests passed.

3. RAG Engine Tests

Focused on verifying that the core RAG assembly process works correctly:

- Context selection (top-k chunks)
- Prompt construction including system + user + context blocks
- OpenAI API mocks confirm generation requests use the right structure
- Response evaluation confirms citation insertion when context is used

Result: All RAG engine tests passed.

4. User Interface Tests

Tested Streamlit UI behavior through mock interactions:

- Empty input detection
- Disabling “Ask” button until knowledge base is built
- Ensuring API keys never appear in plaintext
- Confirming error messages appear for missing embeddings or missing .env

Result: All UI tests passed.

Unit Testing Summary

Component	Number of Tests	Pass Rate
Data Collector	2	100%
Embedding/Vector DB	3	100%
RAG Engine	3	100%
UI Guards	2	100%
Total	10 Tests	100% Passed

- This confirms that core components behave reliably and regressions are caught early during development.

4. Test Case Summaries

Test Case ID	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC01	Error Handling for Missing Dependency (bs4)	bs4 NOT installed	Launch Streamlit app	Error message appears, installation required	Correct error shown; pip install fixed issue	Pass
TC02	PDF-to-Text Ingestion	PDF in data_sources/	Run pdf_to_text.py	PDF converted to .txt	PDF successfully converted; no errors	Pass
TC03	Accurate Retrieval for Threat Modeling	QA data loaded/chunked	Enter "What is threat modeling?" in UI; check retrieved chunks	Relevant chunk returned; high token overlap	Achieved 77% token overlap, 100% sentence attribution	Pass
TC04	Accurate Response to Symmetric vs Asymmetric Q.	QA data loaded	Enter "What is the difference between symmetric and asymmetric encryption?"	Technically correct answer with supporting reference	Answer was complete, referenced context, aligned with KB	Pass

Test Case ID	Test Scenario	Preconditions	Test Steps	Expected Result	Actual Result	Status
TC05	Performance: Query Response Latency	KB and models loaded	Send queries; measure response times	<10s per response	Avg 16.29s/response	Partial Pass

5. Conclusion

The Cybersecurity RAG system is fully functional and performs well across accuracy, reliability, and usability dimensions. Aside from a single missing dependency (bs4), no system-blocking issues were encountered.

Unfinished SRS Items

Explanation and future directions:

- **Real-time threat intelligence integration:** Deferred due to API availability and scope. Would require aggregation of external feeds and live context updates.
- **Multi-language support:** Out of scope for current demo; future work could explore multilingual embedding and response pipelines.
- **Advanced admin functions and analytics:** Basic logging in place; future implementation will add dashboards, audit trail, and admin role-based controls.
- **Distributed KB storage/database:** Current solution sufficient for small to mid-sized cohorts; FAISS, Pinecone, or other scalable backends recommended for expansion.

4. CONCLUSION & FUTURE WORK

1. Summary of the Project

This project addressed the challenge of delivering accurate, contextualized cybersecurity education using AI-driven question answering. We built a modular Retrieval-Augmented Generation (RAG) system that integrates document ingestion, semantic search, neural reranking, and LLM-based response generation. The methodology ensured that responses were not only relevant but also grounded in authoritative cybersecurity resources, meeting diverse user needs from novices to advanced practitioners. Key takeaways include the value of automated context selection, the importance of accurate response grounding, and the efficacy of hybrid retrieval with generative AI for educational platforms.

2. Lessons Learned

- **Technical Skills:**

- Developed expertise in modern NLP toolchains (sentence-transformers, cross-encoder, OpenAI GPT APIs).
- Learned to build robust data pipelines for document processing and semantic search.
- Practiced optimizing query processing for both speed and accuracy.
- Explored effective visualization of NLP performance metrics.

- **Soft Skills:**

- Improved project management and iterative code development.
- Practiced collaborative debugging, modular architecture, and thorough documentation.

- Refined teamwork and communication, especially in integrating multiple system components under tight timelines.
- **Cybersecurity Perspective:**
 - Understood crucial security best practices around input validation, secure API key storage, and ethical content filtering.
 - Implemented real-world security protocols and integrations, such as alignment with OWASP and NIST.
 - Gained experience in addressing actual security challenges in prompt design and data ingestion.
- **Computer Science Perspective:**
 - Gained insight into algorithm design for efficient semantic retrieval.
 - Practiced computational optimization for embedding, search, and reranking.
 - Refined the modular design to easily swap or update algorithms and interfaces.
- **Information Systems Perspective:**
 - Understood and implemented sound data management and source integration strategies for scalable KB construction.
 - Designed for extensible business processes in educational technology, including robust content ingestion and batch updates.

3. Future Work

❖ Technical Improvements:

- Integrate multi-user features and fine-grained access control for admin and analytics dashboards.

- Expand to multi-language support and mobile/web deployment for broader accessibility.
- Move to scalable vector DB solutions for knowledge base growth (e.g., FAISS, Pinecone).
- Develop additional security filters and real-time threat intelligence integrations.
- Add automated QA evaluation, such as RAGAS or more advanced answer scoring tools.

❖ **Practical/Business Enhancements:**

- Integrate with learning management systems (LMS) for seamless classroom adoption.
- Provide richer data analytics and reporting for educators and administrators.
- Develop pathways for community contribution of new content or modules.
- Refine onboarding and educational content structuring for even more effective learning outcomes.

❖ **Recommendations for Next Team:**

- Ensure continual testing as new models or KB sources are added.
- Prioritize ethical and privacy standards for all user and admin interactions.
- Maintain open documentation and system modularity for rapid evolution

5. REFERENCES

- OpenAI API Documentation
- Sentence Transformers Documentation
- Cross-Encoder MS MARCO Documentation

- pdfplumber and pypdf Documentation
- Streamlit Documentation
- OWASP and NIST Frameworks
- IEEE SRS Standard

Appendix

A. requirements.txt

Include the full content of your project's requirements file for environment reproduction:

text

streamlit

plotly

python-dotenv

sentence-transformers

torch

openai>=1.0.0

pdfplumber

pypdf

beautifulsoup4

lxml

numpy

B. .env Configuration Example

Sample .env file (do not share actual API keys in your final public report):

text

OPENAI_API_KEY=sk-xx

C. Screenshots and Supporting Images

Include labeled, captioned screenshots demonstrating:

- PDF-to-text conversion logs (showing successful ingestion)
- Chunking and embedding statistics/dashboard
- Streamlit UI: initial query panel, response cards, context display
- Retrieval and reranking results (top documents for a query)
- Metric gauges and comparison tables (RAG vs ChatGPT)
- Error handling or security filter messages (blocked queries)
- Admin/configuration panels or KB analytics (if implemented)

D. Pseudocode Samples

Provide the main pseudocode blocks used in your report, e.g.:

- Query Answering Pipeline
- PDF Ingestion and Chunking
- Security Filtering Logic

E. Glossary of Technical Terms

Brief explanations for each acronym or technical term:

- **RAG** – Retrieval-Augmented Generation
- **LLM** – Large Language Model
- **Embedding** – Vectorized representation of text for semantic search
- **Cross-Encoder** – Neural reranker for search results
- **OWASP** – Open Web Application Security Project
- **NIST** – National Institute of Standards and Technology

- **Prompt Injection** – Attack on LLM via crafted prompts
- **Token Overlap/Bigram F1/Sentence Attribution** – Metrics for answer grounding/fidelity
- **Streamlit** – Python web app framework
- **pdfplumber** – PDF-to-text extraction library
- **pypdf** – PDF file manipulation library

F. Batch Logs/Data Samples (If applicable)

- Example system logs showing ingestion results, errors handled, and chunk statistics.
- Metric output CSV or JSON for QA evaluations.

G. Contributor Acknowledgements

- List of team members, roles, major code/contribution areas.
- Any outside collaborators or external guidance received.

H. Troubleshooting Notes and Solutions

1. Environment Setup Issues

Problem:

- `ModuleNotFoundError` or import errors on Streamlit, OpenAI, Sentence Transformers, etc.

Solution:

- Ensure Python 3.8+ is installed (`python --version`).
- Install all dependencies with:

`pip install -r requirements.txt`

- If using a virtual environment, activate it before installing:

python -m venv rag_env

source rag_env/bin/activate # or rag_env\Scripts\activate on Windows

pip install -r requirements.txt

2. API Key Problems

Problem:

- Errors such as “API key not found” or “Unauthorized” when generating answers.

Solution:

- Create a .env file in your project root with:

OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxx

- Never hard-code API keys in Python files.
- If rate limiting occurs, check your OpenAI account usage dashboard.

3. PDF/Text Ingestion Errors

Problem:

- UnicodeDecodeError or corrupted text output when processing some PDFs.

Solution:

- Your scripts already use errors='ignore' on file open for robustness.
- If a file fails to process, check its encoding and try manual conversion.
- For unsupported PDFs, open or convert first with a PDF editor.

4. Streamlit/UI Crashes or Fails to Start

Problem:

- App fails to launch, crashes, or modules do not load.

Solution:

- Run Streamlit from the correct directory.
- Restart terminal or IDE to clear cached states.
- Review and correct any import or indentation errors.

5. Model and Embedding Problems

Problem:

- Sentence Transformers or Cross-Encoder models fail to download.

Solution:

- Ensure stable internet connection during first launch.
- If behind a proxy/firewall, set appropriate network settings or download models manually.
- Upgrade pip and retry installation:

pip install --upgrade pip

pip install sentence-transformers

6. OpenAI API Rate Limiting

Problem:

- Errors about hitting OpenAI usage limits.

Solution:

- Check your usage dashboard at OpenAI.
- Wait until quota resets, or upgrade your account if needed.
- Add retry logic (with exponential backoff) if automating batch queries.

7. Security/Filtering Issues

Problem:

- Legitimate queries are blocked, or dangerous prompts are not filtered.

Solution:

- Review filtering patterns in input validation logic.
- Refine security check pseudocode to whitelist safe educational queries and block known exploitative patterns.
- Periodically update blocklist/allowlist based on logs and use cases.

8. Performance and Scaling Issues**Problem:**

- Slow response times when retrieving or embedding many chunks.

Solution:

- Limit chunk size and number of returned results.
- Use batch embedding and retrieval where possible.
- For very large KBs, consider using FAISS or Pinecone for scalable vector search.

General Advice:

- Always check logs for detailed error messages.
- Document all non-trivial fixes or recurring issues for future project cycles.
- Consult official docs for each library/tool for specific platform- or version-related guidance.

This Troubleshooting section will help peer students and instructors quickly resolve the most common deployment and runtime issues, supporting smooth project adoption and extension.